

HyTrig: A Model Checker for Bounded Hybrid Games

Qais Hamarneh

Department of Informatics

Karlsruhe Institute of Technology

Karlsruhe, Germany

qais.hamarneh@kit.edu

Maike Schwammberger

Department of Informatics

Karlsruhe Institute of Technology

Karlsruhe, Germany

schwammberger@kit.edu

Abstract—Hybrid games provide a powerful modelling framework for embedded and cyber-physical systems, but their mixed discrete–continuous dynamics make them challenging to analyse. Bounded hybrid games yield expressive decidable fragments compared to systems with infinite semantics, yet tool support for their verification remains limited. We present HyTrig, a new model checker and simulation tool for bounded hybrid games. HyTrig offers a practical toolchain for specifying and analysing hybrid game models using Alternating-Time Temporal Logic (ATL) extended with real-valued constraints over bounded scenarios. It implements a novel model checking algorithm for Hybrid Games with Triggers (HGT), a framework that incorporates agents’ rationale directly into the game model. This trigger-based discretisation makes model checking decidable while preserving the full power of both the discrete and continuous dynamics of the system. HyTrig is implemented in the programming language Julia, taking advantage of its high performance for scientific and numerical computing. Its graphical interface allows users to inspect models, explore generated winning strategies, or analyse counterexamples produced during verification. Overall, HyTrig establishes a strong foundation for supporting additional decidable fragments, such as monotonic hybrid games and games with finite traces.

Index Terms—Bounded Hybrid Systems, Hybrid Games, Model Checking, Hybrid ATL, Hybrid Games with Triggers.

I. INTRODUCTION

Cyber-physical systems, such as autonomous vehicles and surgical robots, operate at the intersection of discrete control and continuous physical changes. When these systems interact, hybrid games [1] provide a powerful framework for modelling their strategic behaviour. The analytical cost of this expressiveness is high: the combination of unbounded continuous evolution and adversarial choices typically results in infinite branching, rendering model-checking techniques ineffective [2].

Even when restricted to bounded behaviours, hybrid systems remain generally undecidable [3]. Nonetheless, bounded semantics admit more expressive decidable fragments [4] and preserve much of the modelling power of hybrid systems, making several otherwise undecidable fragments [3], [5] amenable to algorithmic verification. Despite this, tool support for analysing bounded hybrid games, especially when reasoning about strategic properties, remains severely limited.

In this paper, we focus on a specific fragment of hybrid games that is decidable in bounded semantics, namely *Hybrid Games with Triggers (HGT)* [5]. HGT addresses one of the central sources of undecidability in hybrid games: the agents’ freedom to choose arbitrary times to act. Triggers encode each agent’s rationale for acting as a set of conditions over the continuous state, effectively discretising the infinite timing choices without restricting the underlying continuous or discrete dynamics. In bounded settings, this yields a powerful, decidable fragment of hybrid games that remains expressive enough for realistic tasks.

The HGT model was introduced from a purely theoretical perspective in [5], leaving several semantic edge cases, such as deadlock conditions, unresolved in the absence of an operational implementation. In this paper, we close this gap with three contributions: a bounded semantics for HGT, paired with an extension of alternating-time temporal logic (ATL) [6] with real-valued constraints, which we call *Hybrid ATL*; a model-checking algorithm for HGT; and HyTrig¹, a model-checking tool built on the hybrid game model of [1] as a multi-agent extension of hybrid automata [7].

Our key contributions in this paper are the following:

- a) **Bounded Semantics for HGT and Hybrid ATL:** A comprehensive bounded semantics for Hybrid Games with Triggers (HGT) that explicitly resolves semantic edge cases such as deadlock conditions, paired with Hybrid ATL, an extension of alternating-time temporal logic with real-valued constraints to formally express winning conditions.
- b) **Model-Checking Algorithm for HGT:** A novel model-checking algorithm that traverses the game progression to verify the existence of winning strategies under Hybrid ATL over HGT models.
- c) **HyTrig:** A powerful, open-source model-checking tool engineered specifically for bounded hybrid games, supporting multi-agent extensions of hybrid automata via both graphical and command-line interfaces.

We position HyTrig within the landscape of existing verification tools in Sect II. Sect. III establishes the theoretical foundations, defining the underlying hybrid game model, Hy-

¹<https://gitlab.com/hgt-mc-group/HGT-MC-project>

brid Games with Triggers III-A and Hybrid ATL III-B. We introduce the HyTrig tool architecture in Sect. IV. The model checking algorithm is detailed in Sect. V and its complexity and correctness are proven in Section VI. Sect. VII demonstrates the tool's practical application through multiple use-cases and benchmark examples, before Sect. VIII concludes with a discussion of planned extensions.

II. RELATED WORK

Model-checking algorithms for Alternating-time Temporal Logic (ATL) [6] have been implemented in frameworks such as MOCHA [8], MCMAS [9], PRISM-games [10], and ATL*AS [11]. These tools target finite-state multi-agent systems and verify ATL (or probabilistic ATL) properties through exhaustive or symbolic exploration of the underlying state space, an approach that is effective precisely because the state space is finite.

Extending such techniques to real-time systems with infinite state spaces requires a property-independent finite abstraction, such as the clock regions defined for timed automata [12]. Tools like UPPAAL Tiga [13] and UPPAAL Stratego [14] build on such abstractions but, rather than constructing them upfront, implement efficient on-the-fly algorithms to solve timed games and weighted timed games, respectively, bypassing full state-space exploration.

UPPAAL Tiga leverages the structural decidability of timed-game model checking, while UPPAAL Stratego instead employs reinforcement learning to learn near-optimal strategies. However, UPPAAL Stratego restricts continuous dynamics to reward variables that cannot influence the game dynamics, i.e., they cannot appear in guards or invariants.

Beyond timed automata, hybrid systems with more general continuous dynamics face a more fundamental obstacle. Reachability becomes undecidable even for relatively simple classes of hybrid systems [2]. Decidable fragments have been obtained by restricting either the discrete [1], [15] or continuous dynamics [16], [17] so that the system admits a finite abstraction [18]. Similar restrictions yield decidable fragments of time-bounded hybrid systems [3]. Differential dynamic (resp. game) logic [19], [20] supports deductive verification using theorem provers such as KeYmaera X [21], [22]. In contrast, hybrid games with triggers (HGT) [5] provide a finite abstraction for bounded hybrid games by incorporating agents' rationale, without restricting either the discrete or continuous dynamics.

Even within decidable fragments, scalability remains a significant and persistent challenge for verification. HyTech [23] pioneered the automated analysis of linear hybrid automata. CheckMate [24] and PHAVer [25] improve scalability through a range of advanced approximation techniques. SpaceX [26] supports efficient and precise reachability computation, and C2E2 [27] employs progressively refined over-approximations to reason effectively about bounded systems. Non-linear dynamics are handled by tools such as dReach [28], based on δ -decision procedures, while Chen et al. [29] use Taylor-model-based approaches. Within the Julia ecosystem, JuliaReach [30] provides a high-performance framework for

reachability, which HyTrig indirectly benefits from through Julia's numerical performance.

Unlike approximation-based approaches, our model checking algorithm performs an exploration of the reachable state space, discretised through triggers. This approach avoids the need for over-approximation and enables very precise reasoning within bounded systems. Although the worst-case complexity remains very high, the algorithm often allows verification by exploring only a small fraction of the state space. Trigger-based discretisation [5] requires agents to act with precise timing, which limits HGT's applicability to autonomous agents rather than human or natural actors, whose behaviour is better captured by threshold- or tolerance-based decisions.

While existing ATL and game-based tools largely restrict continuous dynamics, and existing hybrid-system tools largely target single-agent reachability rather than strategic reasoning, HyTrig supports strategic, adversarial verification over hybrid games with unrestricted continuous and discrete dynamics.

III. THEORETICAL FOUNDATIONS

As a model checker for bounded hybrid games, HyTrig relies on two core components: the formal model of bounded hybrid games with triggers (HGT) and an extension of ATL with real-valued constraints, which we call Hybrid ATL. Central to the following definitions is a set of real-valued variables $X = \{x_1, x_2, \dots, x_n\}$, representing quantities that may evolve over time. These could be spatial position, speed, acceleration, temperature, or other values that change continuously or discretely during the game's execution. *Expressions* and *constraints* are used to define both the hybrid game model and the Hybrid ATL properties that HyTrig verifies. HyTrig uses standard programming-language syntax (e.g., $\&\&$ for conjunction and $\parallel$ for disjunction). For clarity, however, we present the formulae here using conventional logical notation.

Definition III.1 (Expressions and Constraints). An Expression θ in HyTrig is an arithmetic or geometric expression over the set of real-valued variables in X

$$\theta, \theta' := x \mid c \mid -\theta \mid \theta * \theta' \mid f(\theta, \dots, \theta'),$$

with $x \in X$ a variable, $c \in \mathbb{R}$ a constant, $*$ an arithmetic binary operator, and f a function of arbitrary arity, e.g., the currently implemented functions are $\{\sin, \cos, \tan, \cot, \min, \max, \dots\}$. The set of all expressions over X is denoted Θ_X .

Constraints φ in HyTrig are quantifier-free predicates over X

$$\varphi, \varphi' := \text{true} \mid \theta \sim \theta' \mid \neg\varphi \mid \varphi \wedge \varphi',$$

where $\sim \in \{<, \leq, >, \geq, =, \neq\}$. The set of all constraints over X is denoted by $\mathbb{C}_X$. The or, $\vee$, and imply, $\rightarrow$, operators are defined in the usual way.

The values of expressions and constraints are determined by the valuation of the variables in X . A valuation $\nu : X \rightarrow \mathbb{R}$ is a function that assigns each variable in X a real number. The set of all valuations of the variables in X is denoted by $\text{Val}(X)$. We write $\nu(\theta)$ for the value of an expression θ under

a valuation ν , obtained by replacing each variable $x \in X$ with $\nu(x)$. Likewise, we write $\nu \models \varphi$ to indicate that the constraint φ holds under ν , and $\nu \not\models \varphi$ otherwise.

The values of the variables in X *evolve* during the execution of a hybrid game, either discretely or continuously.

Continuous evolution occurs according to a set of differential equations called a *flow* $= \{x'_i = \theta_i \mid x_i \in X\}$. We write $(\nu + t)_{flow}$ to denote the valuation obtained by evolving the variables in X according to *flow* for a duration $t \in \mathbb{R}_{\geq 0}$ starting from the valuation ν .

Discrete evolution occurs according to a discrete assignment $x := \theta$ for some $x \in X$, and the valuation ν evolves so that $\nu[x := \theta](y) = \nu(\theta)$ if $y = x$ and $\nu(y)$ otherwise. This definition extends naturally to an ordered set of discrete assignments $A = \{x_{i_1} := \theta_1, \dots, x_{i_m} := \theta_m\}$, where

$$\nu[A] = \nu[x_{i_1} := \theta_1] \dots [x_{i_m} := \theta_m].$$

The discrete evolution happens when an *agent* in a hybrid game takes an *action*, causing the discrete, sudden evolution. In hybrid games with triggers, the agents' rationale for taking an action is expressed using *triggers*. For a bouncing ball, the trigger is hitting the floor. For an autonomous car, it could be reaching an intersection, *triggering* the car to set a turn signal, or getting close to a vehicle ahead, *triggering* the car to slow down.

Definition III.2 (Trigger). A trigger is a constraint $tr \in \mathbb{C}_X$ whose solution set is closed under the normal topology of $\mathbb{R}^{|X|}$. Thus, given any set of differential equations *flow* and any valuation ν over the variables in X :

- there exists a time $t \in \mathbb{R}_{\geq 0}$, called *time-to-trigger*, such that:
 $(\nu + t)_{flow} \models tr$ and for all $t' \in [0, t)$, $(\nu + t')_{flow} \not\models tr$, or
- $(\nu + t)_{flow} \not\models tr$ for all $t \in \mathbb{R}_{\geq 0}$.

The set of all triggers over X is denoted by $\mathbb{T}_X$.

We write $t_{tt}(flow, \nu, tr)$ (short for *time-to-trigger*) to denote the minimal time to satisfy a trigger starting from the valuation ν and evolving according to *flow*. The function t_{tt} returns ∞ if the trigger is never satisfied.

Example. We showcase the concept of triggers using the classic hybrid systems example of a bouncing ball (shown in Fig. 1). Here, the trigger is when the ball hits the ground, i.e. $h = 0$. Starting from initial motionless state $\nu_0(spd) = 0 \text{ m/s}$ and $\nu_0(h) = 10 \text{ m}$, the ball falls according to the gravity constant $g = -9.8 \text{ m/s}^2$:

$$flow = \{h' = spd, \quad spd' = -9.8\}.$$

The *time-to-trigger* determines when the ball reaches its trigger—when it hits the ground—according to the equations of motion:

$$t_{tt}(flow, \nu_0, h = 0) = 1.428 \text{ s}.$$

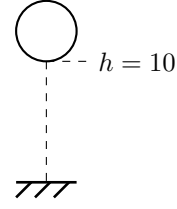


Fig. 1: Bouncing ball starting at height $h = 10$.

A. Hybrid Games with Triggers (HGT)

Hybrid games [1] specify the discrete and continuous dynamics of the variables and how agents' actions influence these dynamics. Hybrid games with triggers (HGT), as defined in [5], offer a unique approach to decidability by eliminating the infinite choice of when actions can be executed, instead basing this choice on a finite set of conditions that make the agents take the action, the agents' triggers. The addition of triggers makes model checking bounded hybrid games decidable without restricting the game's continuous or discrete dynamics.

The definition of HGT used in HyTrig differs from the one used in [5] in that the HGT progresses sequentially rather than concurrently. The decision to use a sequential progression was made to align with the common definition of hybrid games in the literature and to make defining an HGT easier for HyTrig users.

Definition III.3 (Hybrid Games with Triggers). A hybrid game $\mathcal{G}$ is a tuple

$$\mathcal{G} := (X, \nu_0, Agt, Act, Loc, l_0, E, trig), \quad (1)$$

where:

- $X = \{x_1, x_2, \dots, x_n\}$ is a set of real-valued variables.
- $\nu_0 \in Val(X)$ is the initial valuation.
- $Agt = \{\alpha, \beta, \dots\}$ a finite, nonempty, set of agents.
- $Act = \{a, b, \dots\}$ is a finite, nonempty set of agents' actions.
- Loc is a finite nonempty set of locations, with $l_0 \in Loc$ as the initial location. Each location $l \in Loc$ consists of:
 - an invariant $inv_l \in \mathbb{C}_X$ which constrains the values the variables may take while the game is in location l .
 - a $flow_l : X \rightarrow \Theta_X$, a continuous transition relation defining the finite (possibly empty) set of differential equations that govern how valuations evolve over time at location l .
- $E \subseteq Loc \times (Agt \times Act) \times \mathbb{C}_X \times (X \rightarrow \Theta_X) \times Loc$ is a finite set of edges representing the discrete transition relation. An edge $e = (l, (\alpha, a), \varphi, Jump, l')$ is defined as follows:
 - l is the source location and l' is the target location.
 - $(\alpha, a) \in Agt \times Act$ is the action a taken by the agent α causing the game to progress according to the edge e . We say agent α is the *owner* of the edge e .
 - φ is the *guard* of the edge.
 - $Jump_e : X \rightarrow \Theta_X$ is the *jump* of the edge, consisting of a finite (possibly empty) ordered set of discrete assignments.

- $trig : Agt \rightarrow \mathcal{P}(\mathbb{T}_X)$ is a function assigning each agent a finite set of triggers. The set $Trig$ contains the triggers of all agents:

$$Trig := \bigcup_{\alpha \in Agt} trig(\alpha).$$

Note that using a pair (*agent*, *action*) in edges is uncommon in the hybrid games literature, where it is more typical to assign each agent its own set of actions. We adopt the paired formulation because it is more concise, and in many practical examples, the agents share the same action set, making a global action set both natural and convenient.

The **semantics** of an HGT is interpreted over a *labelled* transition system. A *configuration* is a pair $\langle l, \nu \rangle$ consisting of a location $l \in Loc$ and valuation $\nu \in Val(X)$. The set of all configurations $Config_{\mathcal{G}}$ of a game $\mathcal{G}$ comprises the states of the transition system, with *initial configuration* $\langle l_0, \nu_0 \rangle$. Two types of transitions exist in the semantics of an HGT, reflecting the continuous and discrete evolution of the game:

Continuous (time) transition $\langle l, \nu \rangle \xrightarrow{t} \langle l, \nu' \rangle$ for $t \in \mathbb{R}_{\geq 0}$, where $\nu' = (\nu + t)_{flow_l}$. This transition is *valid* iff for all $t' \in [0, t]$, $(\nu + t')_{flow_l} \models inv_l$.

Discrete (action) transition $\langle l, \nu \rangle \xrightarrow{e} \langle l', \nu' \rangle$ for an edge $e \in E$, where $\nu' = \nu[Jump_e]$. An action transition is *valid* iff the edge e is *enabled*. An edge $e = (l, (\alpha, a), \varphi, Jump, l') \in E$ is enabled at the configuration $\langle l, \nu \rangle$ when $\nu \models inv_l$, $\nu \models \varphi$, and $\nu[Jump_e] \models inv_{l'}$. We also say the action a is enabled for the agent α . We assume that at any configuration $\langle l, \nu \rangle$, at most one edge $e = (l, (\alpha, a), \varphi, Jump, l') \in E$ is enabled for an agent-action pair (α, a) .

In an HGT, at the start of the game and after every discrete transition, every agent $\alpha \in Agt$ selects a trigger $tr \in trig(\alpha)$ and an action $a \in Act$ to take as soon as the trigger tr is satisfied. The pair (tr, a) is called the *decision* of agent α . A decision $(tr, a) \in trig(\alpha) \times Act$ is called *valid* for agent $\alpha \in Agt$ at the configuration $\langle l, \nu \rangle$ iff both the time transition

$$\langle l, \nu \rangle \xrightarrow{ttt(flow_l, \nu, tr)} \langle l, \nu'' \rangle$$

and the following action transition

$$\langle l, \nu'' \rangle \xrightarrow{(\alpha, a)} \langle l', \nu' \rangle$$

are valid for some valuations ν' and ν'' and some location l' . An agent α can only select a valid decision if a valid decision is available for the agent α . Otherwise, the agent does not select any decision. The HGT progresses according to the selected decision with the *earliest* trigger, i.e., with the smallest *time-to-trigger* at the current configuration. If multiple decisions have triggers which are satisfied at the same time, the action to be executed is selected non-deterministically from the decisions with satisfied triggers.

The following example describes a realistic application of HGT to autonomous driving and serves as a running example throughout the following sections.

Example. We investigate autonomous cars navigating a one-way multi-lane highway as an HGT (cf. Fig. 2). A number of cars $C_1, \dots, C_N$ drive on an L-lane one-way road. Each car moves longitudinally under standard kinematic dynamics while

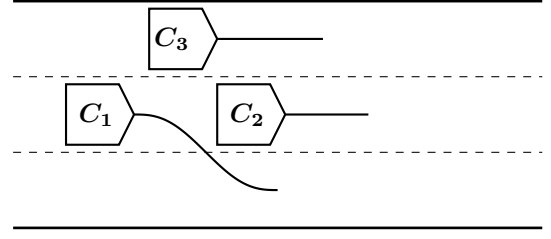


Fig. 2: 3 Cars navigating one-way multi-lane highway. The projected lines show C_1 manoeuvres to the lower lane to overtake the slower C_2 car, while C_2 and C_3 drive straight forward. The lane change manoeuvre projection follows the speed flow of Equation 2.

lateral motion follows a physically smooth quintic minimum-jerk profile [31] that moves the car one lane to the left or right over τ seconds. Cars are assigned different maximum speeds, causing faster cars to change lanes in order to overtake slower ones. Initially, all cars are placed at safe locations: no two cars are colliding or on an unavoidable collision course, and all speeds are within the speed limits.

Variables. Each car i requires seven variables: longitudinal position x_i and speed vx_i , signed acceleration acc_i , lateral position y_i and speed vy_i , a lane change timer τ_i , a lane change direction flag $lc_dir_i \in \{-1, 0, 1\}$.

Agents and Actions. Each of the N cars is an agent. Each car can choose from six actions: *acc*, *dec*, *cruise* (to accelerate, decelerate, or set acceleration to zero), *left* and *right* (to begin a lane change manoeuvre), and *end_lc* (to stop the lane change manoeuvre).

Flows and Invariants. Longitudinal dynamics follow standard constant-acceleration kinematics: $x'_i = vx_i$ and $vx'_i = acc_i$. Lateral dynamics implement the quintic minimum-jerk profile:

$$vy'_i = lc_dir_i \cdot \left(\frac{200}{9} \tau_i - \frac{200}{9} \tau_i^2 + \frac{400}{81} \tau_i^3 \right). \quad (2)$$

This guarantees zero lateral velocity at the start and end of every lane change. Figure 3 shows the lateral speed profile for $\tau = 3$ s. The location invariant constrains all speeds to $[0, v_{max,i}]$ and all lane change timers to $\tau_i \leq \tau$, preventing unbounded manoeuvres.

Jumps. Discrete assignments update acc_i (for longitudinal actions), or set lc_dir_i , and reset τ_i (for lane change actions).

Triggers. Each car has four triggers corresponding to recognisable real-world driving events:

Tailgating trigger fires when a car closes to within a safety distance behind another car ahead in the same lane, prompting the car to slow down, change lanes, or both.

Upper speed-limit trigger fires when an accelerating car reaches its maximum speed $v_{max,i}$, and the lower speed-limit trigger fires when a decelerating car reaches speed 0.

Lane change termination trigger fires once the car has completed its lane change manoeuvre ($\tau_i \geq \tau$) or has become too close laterally to another car.

These four events encode the moments at which a driver, or an autonomous controller, must make a discrete decision.

Trigger-based progression modifies the standard notion of a deadlock. In a typical hybrid system, a configuration is

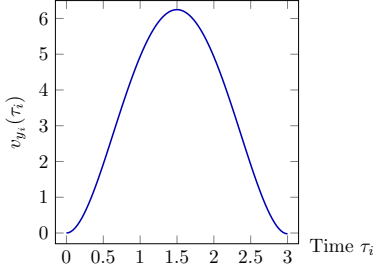


Fig. 3: Lateral speed during a lane change by car C_i for total lane change time $\tau = 3$ s according to Eq. 2.

deadlocked when no time or action transition is available. In an HGT, a configuration can also be deadlocked when an action transition exists but is blocked by the trigger-based progression. We formalise this below.

Internal Variables: We assume three variables *step*, *time* and *location-time* to be in the set of variables X . *step* counts the number of discrete transitions taken; *time* records the global elapsed time; and *location-time* records the time the game spends in its current location. All three variables are initialised to zero:

$$\nu_0(\text{step}) = \nu_0(\text{time}) = \nu_0(\text{location-time}) = 0.$$

time and *location-time* evolve continuously with $\text{flow}_l(\text{time}) = \text{flow}_l(\text{location-time}) = 1$ for all $l \in \text{Loc}$. *time* is never discretely updated, while *location-time* is reset to 0 on every edge $e \in E$ via $\text{Jump}_e(\text{location-time}) = 0$. *step* does not evolve continuously ($\text{flow}_l(\text{step}) = 0$ for all $l \in \text{Loc}$) and is incremented by every edge $e \in E$ via $\text{Jump}_e(\text{step}) = \text{step} + 1$.

Definition III.4 (Deadlock). An HGT configuration $c = \langle l, \nu \rangle$ is *deadlocked* if no valid time transition exists from c , and for every agent $\alpha \in \text{Agt}$ and every trigger $tr \in \text{trig}(\alpha)$, at least one of the following holds:

- i. there is no enabled edge $e \in E$ with $\text{owner}(e) = \alpha$;
- ii. $\nu \models tr$; or
- iii. $\nu \models tr$, but there exists $0 < t \leq \nu(\text{location-time})$ such that $(\nu - t)_{\text{flow}_l} \models tr$, where $(\nu - t)_{\text{flow}_l}$ is an abuse of notation denoting the valuation t time units prior to ν along the flow of l . This is sound in this case because we know that $t < \nu(\text{location-time})$, meaning we know that the game started more than t time units ago and for the last t time units was in the same location l .

Condition (iii) captures the case where the trigger tr was already satisfied earlier in the current location; since HGT permits an action transition only at the *first* moment a trigger becomes satisfied, the window for that transition has passed, and no action can be taken.

To illustrate this concept, we consider the following example.

Example. Consider the single-agent HGT shown in Fig. 4. Without triggers, a deadlock is never reached: if $\nu(x) = 2$ the guard is satisfied and the agent can always take the edge. How triggers can affect deadlocks is illustrated by the following two cases.

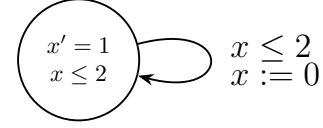


Fig. 4: An HGT illustrating how the choice of trigger set determines whether a deadlock configuration is reachable.

- *Trig* = $\{x \geq 1\}$. The trigger $x \geq 1$ is first satisfied when $\nu(x) = 1$. At that moment, there are two options, the agent can take the action transition, or the time transition proceeds until $\nu(x) = 2$, at which point the invariant $x \leq 2$ prevents further time transitions and the trigger can no longer be used to initiate a discrete transition and thus the configuration is deadlocked.
- *Trig* = $\{x \geq 2\}$. This trigger is first satisfied exactly at $\nu(x) = 2$, which coincides with the boundary of the invariant. At this point, no time transition is possible, so the only available transition is the action transition allowed by the trigger. The game, at this point, must proceed through the action transition, resetting x to 0 and allowing time to progress again. A deadlock is, therefore, never reached.

Bounded hybrid games stop when they reach a given *termination condition* or a *deadlock*. A **termination condition** is a predicate over the configuration of the hybrid game that defines when the game must terminate. The configuration that satisfies a termination condition is called *terminal*. We describe the types of termination conditions allowed in HyTrig in Sect. IV. A run of a bounded hybrid game is called a *play*.

Definition III.5 (Play). A play over an HGT is a finite sequence of configurations $(\langle l_0, \nu_0 \rangle, \langle l_1, \nu_1 \rangle, \dots, \langle l_t, \nu_t \rangle)$ such that:

- For all $i \in \{0, 1, \dots, t-1\}$, the configuration $\langle l_i, \nu_i \rangle$ is not deadlocked or a terminal configuration.
- For each $i \in \{0, 1, \dots, t-1\}$, there exists either:
 - a **time transition**: $\langle l_i, \nu_i \rangle \xrightarrow{t} \langle l_{i+1}, \nu_{i+1} \rangle$ for some time $t \in \mathbb{R}_{\geq 0}$, or
 - an **action transition**: $\langle l_i, \nu_i \rangle \xrightarrow{e} \langle l_{i+1}, \nu_{i+1} \rangle$ for some edge $e \in E$, such that:
 - 1) $\nu \models tr$ for some trigger $tr \in \text{trig}(\text{owner}(e))$, and
 - 2) for all $t \in (0, \nu(\text{location-time})]$, $(\nu - t)_{\text{flow}_l} \not\models tr$.
- The final configuration $\langle l_t, \nu_t \rangle$ is either terminal or deadlocked.

A play in a hybrid game is determined by the choices made by agents throughout the game. These choices are formalised as *strategies*.

Definition III.6 (Strategy). A strategy in an HGT $\mathcal{G}$ is a function

$$st : \text{Config}_{\mathcal{G}} \rightarrow \text{Trig} \times \text{Act}.$$

We write $st(c)_{\text{trg}}$ and $st(c)_{\text{act}}$ to refer to the trigger and action components of the decision, respectively. An agent $\alpha \in \text{Agt}$ follows a strategy st if, in every configuration $c = \langle l, \nu \rangle$, the agent chooses the action $st(c)_{\text{act}}$ once the trigger $st(c)_{\text{trg}}$ is satisfied. A strategy st is called *valid* for agent $\alpha \in \text{Agt}$ if, in every configuration c , the strategy returns a valid decision

for the agent α or $\perp$ when no valid decision exists. A *strategy profile* st_A is a set of valid strategies, one for each agent in the *coalition* $A \subseteq \text{Agt}$, a set of agents with a shared objective.

To describe how the game unfolds when each agent in $A \subseteq \text{Agt}$ follows their strategy in st_A , we define the *outcome function* out . The function $out(st_A)$ returns the set of all possible plays that may result when every agent in A follows their strategy in st_A .

Note that, unlike in most deterministic discrete or concurrent games, the set $out(st_{\text{Agt}})$ for some strategy profile st_{Agt} is not necessarily a single play, since when multiple agents attempt to act at the same time, the executed action is chosen non-deterministically.

B. Hybrid ATL

The syntax of Hybrid ATL consists of *state formulae* and *strategy formulae*. Hybrid ATL is inspired by the definition of Timed CTL in UPPAAL [32] and Timed ATL in [6], [33], [34].

A **state formula** ψ is evaluated at a single configuration $\langle l, \nu \rangle$. For example, the formula $l_1 \wedge x = 10$ holds exactly when $l = l_1$ and $\nu(x) = 10$. In contrast, a **strategy formula** ϕ is interpreted over complete plays reachable under some strategy profile. We consider two forms of strategy formulae.

A **reachability formula** $\langle A \rangle \mathbf{F} \psi$ reads: “there exists a strategy profile for the coalition $A \subseteq \text{Agt}$ that ensures every outcome play reaches a configuration satisfying ψ before the game terminates.” Dually, $\llbracket A \rrbracket \mathbf{F} \psi$ reads: “all strategy profiles for the coalition $A \subseteq \text{Agt}$ ensure every outcome play reaches a configuration satisfying ψ before the game terminates.”

A **safety formula** $\langle A \rangle \mathbf{G} \psi$ reads: “there exists a strategy profile for the coalition $A \subseteq \text{Agt}$ that ensures ψ holds at every configuration along every outcome play until the game terminates.” Dually, $\llbracket A \rrbracket \mathbf{G} \psi$ asserts that “all strategy profiles of A maintain ψ throughout every outcome play”.

For instance, $\langle \alpha \rangle \mathbf{F} (x = 1)$ holds if agent α has a strategy st such that every play in $out(st)$ eventually reaches a configuration satisfying $x = 1$ before the game terminates, while $\langle \alpha \rangle \mathbf{G} (x \geq 0)$ holds if α has a strategy that keeps $x \geq 0$ throughout every such play.

We additionally define a distinguished state formula *deadlock*, which holds at a configuration c if and only if the configuration is deadlocked.

Definition III.7 (Syntax of Hybrid ATL). Given a hybrid game $\mathcal{G}$, the Hybrid ATL syntax in HyTrig consists of *state formulae* ψ and *strategy formulae* ϕ

$$\begin{aligned} \psi &:= \varphi \mid l \mid \text{deadlock} \mid \neg\psi \mid \psi \wedge \psi \\ \phi &:= \psi \mid \langle A \rangle \mathbf{F} \psi \mid \langle A \rangle \mathbf{G} \psi \mid \llbracket A \rrbracket \mathbf{F} \psi \mid \llbracket A \rrbracket \mathbf{G} \psi, \end{aligned}$$

where $\varphi \in \mathbb{C}_X$ is a constraint, $l \in \text{Loc}$ is a location, and $A \subseteq \text{Agt}$ is a coalition.

Additionally, we also define the Boolean operators “or” $\vee$ and “implies” $\rightarrow$ in the usual way.

Definition III.8 (Semantics of Hybrid ATL). Given a hybrid game $\mathcal{G}$, for all configurations $\langle l, \nu \rangle \in \text{Config}_{\mathcal{G}}$, the semantics

of a Hybrid ATL state formula ψ are defined inductively over the structure of ψ :

$$\begin{aligned} \langle l, \nu \rangle \models \varphi & \quad \text{iff } \nu \models \varphi \text{ for a constraint } \varphi \in \mathbb{C}_X. \\ \langle l, \nu \rangle \models l' & \quad \text{iff } l = l' \text{ for a location } l' \in \text{Loc}. \\ \langle l, \nu \rangle \models \text{deadlock} & \quad \text{iff } \langle l, \nu \rangle \text{ is deadlocked.} \\ \langle l, \nu \rangle \models \neg\psi & \quad \text{iff } \langle l, \nu \rangle \not\models \psi. \\ \langle l, \nu \rangle \models \psi_1 \wedge \psi_2 & \quad \text{iff } \langle l, \nu \rangle \models \psi_1 \text{ and } \langle l, \nu \rangle \models \psi_2. \end{aligned}$$

The semantics of a strategy formula ϕ are defined inductively over the structure of ϕ and the plays over the game $\mathcal{G}$:

$$\begin{aligned} \mathcal{G} \models \langle A \rangle \mathbf{F} \psi, & \quad \text{iff there exists a strategy profile } st_A \text{ such that for all plays } \lambda \in out(st_A), \text{ there exists a configuration } c_i \in \lambda \text{ such that } c_i \models \psi. \\ \mathcal{G} \models \langle A \rangle \mathbf{G} \psi, & \quad \text{iff there exists a strategy profile } st_A \text{ such that for all plays } \lambda \in out(st_A), \text{ for all configurations } c_i \in \lambda, c_i \models \psi. \\ \mathcal{G} \models \llbracket A \rrbracket \mathbf{F} \psi, & \quad \text{iff } \mathcal{G} \not\models \langle \text{Agt} \setminus A \rangle \mathbf{G} (\neg\psi). \\ & \quad \text{There exists no strategy for the other agents } \text{Agt} \setminus A, \text{ where } \neg\psi \text{ holds globally, i.e. } \psi \text{ must eventually hold.} \\ \mathcal{G} \models \llbracket A \rrbracket \mathbf{G} \psi, & \quad \text{iff } \mathcal{G} \not\models \langle \text{Agt} \setminus A \rangle \mathbf{F} (\neg\psi). \\ & \quad \text{There exists no strategy for the other agents } \text{Agt} \setminus A, \text{ where } \neg\psi \text{ eventually hold, i.e. } \psi \text{ must hold globally.} \end{aligned}$$

Example. Cars in Example III-A (cf. p. 4) must satisfy two properties: *safety* and *reachability*. *Safety* can only be guaranteed if all cars work collaboratively rather than adversarially. We express this by treating all cars as a coalition seeking a joint safe strategy:

$$\langle C_1, \dots, C_N \rangle \mathbf{G} \bigwedge_{1 \leq i < j \leq N} (|x_i - x_j| \geq 5 \wedge |y_i - y_j| \geq 5). \quad (3)$$

Each car must also ensure its forward progression. The following reachability query tests whether each car C_i can advance a distance proportional to its maximum speed within the given time limit:

$$\langle C_i \rangle \mathbf{F} (x_i \geq x_i^0 + 10 \cdot v_{\max, i}). \quad (4)$$

where $x_i^0 := \nu_0(x_i)$.

IV. THE MODEL CHECKING TOOL

HyTrig is a model checker for bounded hybrid games. It is implemented in the programming language Julia² [35], taking advantage of its multi-platform, high-performance runtime and its rich ecosystem of scientific computing libraries, in particular `DifferentialEquations.jl`³ [36]. We provide a general introduction to our tool HyTrig in this section and give more details on the model-checking algorithm in Sect. V.

The tool offers a modern and intuitive graphical user interface (GUI), implemented using the library `QML.jl`⁴. The GUI allows a user to define the game model and Hybrid ATL properties using the simple math-like syntax as defined in the previous section. Upon checking the queries, the user can explore the generated strategies in the GUI and examine

²<https://julialang.org>

³<https://docs.sciml.ai/DiffEqDocs>

⁴<https://github.com/JuliaGraphics/QML.jl>

existing example or counterexample strategies for each Hybrid ATL property if one exists.

HyTrig can also be run from the command line. The HGT, termination conditions, and the Hybrid ATL queries are written in JSON-structured files with a straightforward syntax.

HyTrig supports three **termination conditions**: a *time bound*, a *maximum number of discrete transitions (steps)*, and a *Hybrid ATL state formula*. These conditions provide flexibility in defining when a play ends while still guaranteeing termination. Note that a Hybrid ATL state formula condition means the game stops once the given formula is met. However, since this formula might never hold, this condition must be paired with another termination condition to ensure termination.

We highlight key challenges addressed during the implementation of the HGT and the model checking algorithm:

a) *Avoiding Zeno Behaviour*: Simulating real-time systems requires avoiding Zeno behaviour [37], in which infinitely many discrete transitions occur within a finite duration. In a game setting, this is especially important, as agents could deliberately exploit Zeno behaviour to block time progression. We prevent this by requiring that the game cannot revisit a location no agent can take the same action within less than $\epsilon = 10^{-5}$ time units, ensuring a strictly positive lower bound on the time elapsed between repeated actions by the same agent. This is a conservative condition that excludes some physically admissible behaviours, but provides a simple and uniform termination guarantee.

b) *Numerical Integration of Differential Equations*: Executing the model checking algorithm in Sect. V requires solving many systems of differential equations to simulate time transitions. This is computationally expensive, and numerical errors accumulate across steps. We use the `DifferentialEquations.jl` library³ for Julia with the *Tsitouras 5/4 Runge–Kutta* method [38], configured with a relative and absolute tolerance of 10^{-6} , offering a suitable balance between numerical precision and computational speed.

c) *Tracking Evolving Constraint Valuations*: The differential equation solver must also detect the moment at which a trigger or a reachability property becomes satisfied, or when a safety property or a location invariant is violated. We address this using the `VectorContinuousCallback` interface⁵ provided by `DifferentialEquations.jl`, which allows a vector of real-valued functions to be monitored during integration and interrupts the solver when any of these functions crosses zero. Each constraint of interest is reformulated as a real-valued function whose zero-crossing corresponds exactly to the event being detected, reducing constraint monitoring to a standard zero-crossing problem.

The first version of HyTrig implements a novel model checking algorithm for hybrid games with triggers. Since HGT imposes no restrictions on the discrete or continuous dynamics of the underlying hybrid game, the implementation provides a strong foundation that fully exploits the expressiveness of the expressions and constraints defined in Sect. III. This includes support for simulating both arithmetic and geometric expressions. HyTrig also includes multiple example models,

one of which is discussed in detail in Sect. VII. Verification algorithms for other bounded hybrid game models are already being developed.

V. MODEL CHECKING BOUNDED HGT

Given a bounded HGT $\mathcal{G}$ and a Hybrid ATL formula ϕ , the model checking algorithm determines whether $\mathcal{G} \models \phi$. The algorithm exploits the trigger-based progression to abstract over states in which no trigger is satisfied and hence no action can occur. This abstraction allows the model checker to traverse the *time-abstract game tree* (simply the *game tree*) in discrete steps via a depth-first search variant, terminating as soon as a state satisfying (reachability) or violating (safety) the property is found, largely avoiding exploration of the complete state space. The game tree has the initial configuration as its root, configurations where a trigger is satisfied as internal nodes, and deadlocks or terminal configurations as leaves. The algorithm traverses this tree to construct a *strategy tree*: a subtree representing a strategy of the coalition under consideration, with additional leaves at the earliest configuration satisfying or violating the property.

A. Strategy Tree

A strategy tree over a bounded HGT game $\mathcal{G}$ is constructed based on a strategy Hybrid ATL formula ϕ of the form $\langle\langle A \rangle\rangle \mathbf{F} \varphi$ or $\langle\langle A \rangle\rangle \mathbf{G} \varphi$. A strategy tree represents a strategy profile for the coalition $A \subseteq \text{Agt}$ as a witness that $\mathcal{G} \models \phi$, if one exists. The tree nodes represent a subset of the reachable configurations of the HGT. Not all reachable configurations are represented by a node, and the same configuration might be represented by multiple nodes. We distinguish the following types of nodes in the strategy tree:

- **Decision node**: represents either the initial configuration or a configuration reached by a discrete transition. At this node, each agent $\alpha \in \text{Agt}$ gets to choose a decision $(tr, a) \in \text{trig}(\alpha) \times \text{Act}$, committing to execute action a as soon as trigger tr becomes satisfied.
- **Trigger node**: represents a configuration at which some trigger $tr \in \text{trig}(\alpha)$ for some agent $\alpha \in \text{Agt}$ is satisfied, enabling agent α to execute an action via an edge $e \in E$ with $\text{owner}(e) = \alpha$. The children of a trigger node are decision nodes. If more than one trigger is satisfied at the same configuration, a separate trigger node corresponds to each satisfied trigger. We say agent α is to-blame for the trigger node if the node corresponds to some trigger $tr \in \text{trig}(\alpha)$.
- **Property node**: represents a configuration at which the truth value of the reachability or safety property changes. A property node has no children.
- **Final node**: represents a deadlock or a terminal configuration. A final node has no children.

In the following, we show how a strategy tree can be constructed from a given strategy and a Hybrid ATL formula. The strategy tree prunes the complete game tree in multiple ways: (i) the branches end as soon as the reachability property has been reached, or the safety property violated; (ii) it dismisses all branches based on decisions of the agent following the strategy that differ from the decision suggested by the strategy;

⁵https://docs.sciml.ai/DiffEqDocs/stable/features/callback_functions

and (iii) it eliminates branches based on time transitions longer than the decision suggested by the strategy.

Definition V.1 (Strategy Tree). Let $\mathcal{G}$ be an HGT, let ϕ be a Hybrid ATL formula of the form $\langle\langle\alpha\rangle\rangle\mathbf{F}\psi$ or $\langle\langle\alpha\rangle\rangle\mathbf{G}\psi$ for some agent $\alpha \in \text{Agt}$, and let st be a valid strategy for α . The *strategy tree* induced by st is a subtree of the full game tree, defined inductively as follows.

- **Root.** The initial configuration $\langle l_0, \nu_0 \rangle$ is a decision node and is the root of the strategy tree.
- **Inductive step.** Let dn be a decision node in the strategy tree with associated configuration $c = \langle l, \nu \rangle$, and let $st(c) = (tr_\alpha, a_\alpha)$. If the strategy chooses no decision $\perp$, we consider $\text{ttt}(\text{flow}_l, \nu, tr_\alpha) = \infty$; nothing else changes. The children of dn are determined as follows.

- **Strategy trigger and action (agent α 's move).** Let $c_{\alpha-tr}$ be the configuration reached by the time transition of $c \xrightarrow{\text{ttt}(\text{flow}_l, \nu, tr_\alpha)} c_{\alpha-tr}$, and let $c_{\alpha-a}$ be the configuration reached by the action transition $c_{\alpha-tr} \xrightarrow{(\alpha, a_\alpha)} c_{\alpha-a}$. The strategy tree contains:
 - * a trigger node tn_α for $c_{\alpha-tr}$ as a child of dn , and
 - * a decision node dn_α for $c_{\alpha-a}$ as a child of the trigger node tn_α .

Agent α is to-blame for both nodes tn_α and dn_α .

- **Adversarial interruptions (all other agents).** For every agent $\beta \in \text{Agt} \setminus \{\alpha\}$ and every trigger $tr_\beta \in \text{trig}(\beta)$ such that $\text{ttt}(\text{flow}_l, \nu, tr_\beta) \leq \text{ttt}(\text{flow}_l, \nu, tr_\alpha)$ (i.e. β 's trigger is satisfied no later than α 's chosen trigger):

- * let $c_{\beta-tr}$ be the configuration reached from c by the time transition of $c \xrightarrow{\text{ttt}(\text{flow}_l, \nu, tr_\beta)} c_{\beta-tr}$; a trigger node tn_β for $c_{\beta-tr}$ is added as a child of dn ;
- * for every action $b \in \text{Act}$ for which there exists an enabled edge $e \in E$ with $\text{owner}(e) = \beta$ $\text{act}(e) = (b)$, let $c_{\beta-b}$ be the configuration reached by the action transition $c_{\beta-tr} \xrightarrow{e} c_{\beta-b}$; a decision node dn_β for $c_{\beta-b}$ is added as a child of the trigger node tn_β . The agent β is to-blame for both nodes tn_β and dn_β .

- **Early-termination configurations.** If there exists a time $t \in [0, \text{ttt}(\text{flow}_l, \nu, tr_\alpha)]$ such that the configuration c' reached by the time transition $c \xrightarrow{t} c'$ satisfies any of the following conditions:

- 1) c' is a deadlocked,
- 2) c' is a terminal configuration,
- 3) $c' \models \psi$ and $\phi \equiv \langle\langle\alpha\rangle\rangle\mathbf{F}\psi$, or
- 4) $c' \not\models \psi$ and $\phi \equiv \langle\langle\alpha\rangle\rangle\mathbf{G}\psi$,

then a final node, in case 1) and 2), or a property node, in case 3) and 4), is added for c' as a child of dn . All siblings of c' among the children of dn whose *location-time* valuation is greater than or equal to that of c' are removed from the strategy tree.

The definition extends to strategy profiles st_A for a coalition $A \subseteq \text{Agt}$ and properties of the form $\langle\langle A \rangle\rangle\mathbf{F}\psi$ or $\langle\langle A \rangle\rangle\mathbf{G}\psi$.

Definition V.2 (Winning and Counter Strategies). A strategy profile st_A is *winning* for agents $A \subseteq \text{Agt}$ with respect to a formula ϕ if every leaf node of its strategy tree is:

- a property node satisfying ψ , in the case of the reachability formula $\phi \equiv \langle\langle A \rangle\rangle\mathbf{F}\psi$; or
- a final node satisfying ψ , in the case of the safety formula $\phi \equiv \langle\langle A \rangle\rangle\mathbf{G}\psi$.

For Hybrid ATL formulae of the form $\llbracket A \rrbracket\mathbf{F}\psi$ or $\llbracket A \rrbracket\mathbf{G}\psi$, a winning strategy for the agents $\text{Agt} \setminus A$ with respect to the formulae $\langle\langle \text{Agt} \setminus A \rangle\rangle\mathbf{G}\neg\psi$ or $\langle\langle \text{Agt} \setminus A \rangle\rangle\mathbf{F}\neg\psi$, respectively, is called a *counter* strategy.

In the following, we show how the model checking algorithm, given an HGT and a Hybrid ATL formula, constructs a strategy tree for a winning or a counter strategy if one exists, and returns an empty tree otherwise.

B. Model Checking Hybrid ATL.

Model checking a Hybrid ATL formula proceeds from the initial configuration as a root node, recursively exploring all possible strategy trees until either a winning or a counter strategy is found, in which case the algorithm returns the valuation of the formula and the strategy tree, or until the algorithm verifies that no winning or counter strategy exists, in which case the algorithm returns just the valuation of the formula.

For brevity, we omit the verification procedure for state formulae, as these are handled trivially on an individual configuration basis. Furthermore, the “for all strategies” operators $\llbracket A \rrbracket$ can be reduced to the corresponding “there exists a strategy” operators $\langle\langle A \rangle\rangle$ as follows:

$$\begin{aligned} \mathcal{G} \models \llbracket A \rrbracket\mathbf{F}\psi & \text{ iff } \mathcal{G} \not\models \langle\langle \text{Agt} \setminus A \rangle\rangle\mathbf{G}(\neg\psi) \\ \mathcal{G} \models \llbracket A \rrbracket\mathbf{G}\psi & \text{ iff } \mathcal{G} \not\models \langle\langle \text{Agt} \setminus A \rangle\rangle\mathbf{F}(\neg\psi) \end{aligned}$$

This leads to two cases, the existence of reachability and safety strategies, that must be investigated.

1) *Reachability Case* ($\langle\langle A \rangle\rangle\mathbf{F}\psi$): Algorithm 1 implements the model checking procedure for reachability formulae of the form $\phi \equiv \langle\langle A \rangle\rangle\mathbf{F}\psi$. It takes as input a decision node dn and the formula ϕ , and returns TRUE together with the winning strategy tree rooted at dn if such a strategy exists, or FALSE with only the node dn as a tree otherwise. The algorithm is initially called at the root node corresponding to the initial configuration. Only the branches that contribute to a winning strategy are retained in memory. All losing branches are discarded immediately upon detection.

a) *Base cases.*: The function first checks conditions that terminate the recursion without expanding the game tree further. If the current configuration satisfies ψ , lines 2-5, a property node is attached as the sole child of dn and the function returns TRUE: the reachability objective is met. If the configuration satisfies a termination condition, or is deadlocked, lines 6-7, the function returns FALSE: The objective cannot be met from this configuration.

The function TIMECHILDREN(dn, A), line 10, for a decision node dn and a coalition $A \subseteq \text{Agt}$ returns all trigger, property, and final nodes direct children of dn , i.e., reachable by a valid time transition from the configuration of dn . The list of children is ordered in ascending time, and if two nodes share the same time, then property and final nodes are prioritised

over trigger nodes, and triggers of the agents $Agt \setminus A$ are prioritised over triggers of the agents A . The variable `child-results`, line 9, stores the evaluation results of the most recently explored child of dn as we discuss in the post loop case (Paragraph V-B1c).

b) Iterating over time-children.: The children returned by `TIMECHILDREN` are processed in order. If the child is a property node, line 12-14, i.e. the property is reached, the child is appended to dn children and the algorithm returns `TRUE`.

If the child is a final node, lines 15-16, the algorithm terminates and returns `FALSE` without appending the child.

If the child is a trigger node, we distinguish two cases: a *coalition trigger*, owned by an agent in A , and an *adversarial trigger*, owned by an agent in $Agt \setminus A$. In either case, however, the function `ACTIONCHILDREN(child)` returns a list of decision nodes representing the configurations reached by action transitions over enabled edges owned by the agent to blame for the trigger node, we refer to these decision nodes as action-child in the Algorithm 1. This list is not sorted.

Trigger nodes: to-blame(child) ∈ A.: (lines 20-26) A coalition agent *chooses* which action to execute. The algorithm recurses on each child decision node in turn; as soon as one returns `TRUE`, that decision node is appended to the trigger node's children, and the function returns `TRUE`, since a single winning action suffices. If no action yields a winning continuation, the trigger node contributes no children, and the function continues to the next time-child.

Trigger nodes: to-blame(child) ∉ A.: (lines 28-32) An adversarial agent *chooses* which action to execute, and the coalition must guarantee ψ against all possible choices. The algorithm therefore recurses on every child decision node; each winning action-child is appended to the trigger node's children as it is confirmed. If any action-child returns `FALSE`, the adversary can select that action to prevent the coalition from reaching ψ , so the function returns `FALSE` immediately. Only if all action-children are winning does the adversarial trigger node constitute a safe branch, is appended to the decision node dn children, and the outer loop continues to the next time-child.

c) Post-Loop Case.: If the for-loop terminates without returning final results, we can conclude the following:

- 1) The list of time-children did not include any property or final nodes.
- 2) The list of time-children is not empty, because otherwise, the base case would have found the dn configuration to be deadlocked and terminated already then.
- 3) The last time-child is either
 - a) a coalition trigger node with all action children returning `FALSE`, or
 - b) an adversarial trigger node with all action children returning `TRUE`.

We know from the deadlock Def. III.4 and Example III-A that the game is *forced* to proceed according to the last trigger-node child. Therefore, the algorithm returns the results of the last child (line 40).

2) *Safety Case* ($\langle\langle A \rangle\rangle G \psi$): Checking formulae of the form $\langle\langle A \rangle\rangle G \psi$ is very similar; the only difference is that property-

Algorithm 1 Model Checking the formula $\phi \equiv \langle\langle A \rangle\rangle F \psi$

```

1: function EVALUATE( $dn, \phi$ )
2:   if  $dn.config \models \psi$  then
3:      $dn.children := [\text{PropN}(dn.config)]$ 
4:                                      $\triangleright$  PropN: Property Node
5:   return True
6: else if IS-FINAL( $dn.config$ ) then
7:   return False
8: end if
9:  $child-results := \text{False}$ 
10:  $children := \text{TIMECHILDREN}(dn, A)$ 
11: for child in children do
12:   if child is Property-Node then
13:     APPEND( $dn.children, child$ )
14:   return True
15: else if child is Final-Node then
16:   return False
17: else if child is Trigger-Node then
18:    $action-children := \text{ACTIONCHILDREN}(child)$ 
19:   for action-child in action-children do
20:     if to-blame(child)  $\in A$  then
21:       if EVALUATE(action-child,  $\phi$ ) then
22:         APPEND( $child.children, action-child$ )
23:         APPEND( $dn.children, child$ )
24:       return True
25:     end if
26:      $child-results := \text{False}$ 
27:   else
28:     if not EVALUATE(action-child,  $\phi$ ) then
29:       return False
30:     end if
31:      $child-results := \text{True}$ 
32:     APPEND( $child.children, action-child$ )
33:   end if
34: end for
35: if not IS-EMPTY( $child.children$ ) then
36:   APPEND( $dn.children, child$ )
37: end if
38: end if
39: end for                                      $\triangleright$  Post-Loop Case
40: return child-results
41: end function

```

nodes return `FALSE` and are not appended, whereas final nodes are appended and return `TRUE`. 816
817

The following example illustrates how Algorithm 1 explores the time-abstract game tree to construct a strategy tree for a winning strategy for the HGT defined in Example III-A. It also highlights the different node types and the extent of pruning applied to the underlying game tree. 818
819
820
821
822

Example. In the HGT of Example III-A (initial placement in Fig. 2), C_1 's maximum speed exceeds that of C_2 and C_3 , so the only way for C_1 to cover the required distance $10 \times v_{\max,1}$ within the time limit (Property 4) is to overtake C_2 on the rightmost lane. We add the collision condition 823
824
825
826
827

$$\bigvee_{i \in \{2,3\}} (|x_1 - x_i| < 5 \wedge |y_1 - y_i| < 5)$$

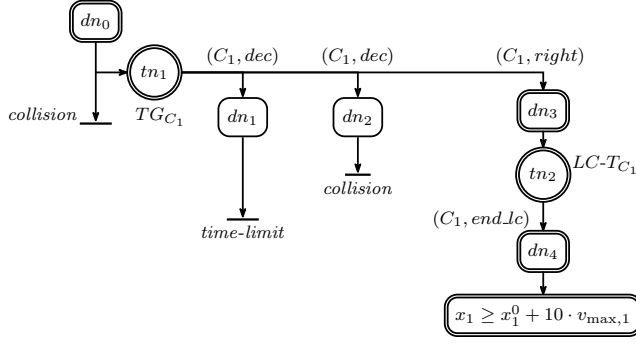


Fig. 5: Game and strategy tree for constructing a winning strategy for C_1 's reachability property (Property 4). Starting from the root decision node dn_0 , through trigger node $tn_{\{1,2\}}$ (circles): the tailgating trigger TG_{C_1} and the lane-change termination trigger $LC-T_{C_1}$. Branches ending in a bar denote final nodes. Double-bordered nodes denote the constructed strategy tree, ending in a property node where C_1 has covered the required distance. Note that the property node is only a part of the strategy tree, not the time-abstract game tree.

as a termination condition, ensuring the resulting strategy is safe. As the cars advance at maximum speed from dn_0 , C_1 's tailgating trigger fires once it nears C_2 , giving C_1 three enabled actions, shown as the action-children of trigger node tn_1 in Fig. 5. Algorithm 1 explores each in turn: *dec* keeps C_1 safe but causes the time limit to be reached before the required distance is covered, so this branch (a final node) is discarded; *left* puts C_1 on a collision course with C_3 , again reaching a final node, and is discarded; *right* keeps C_1 safely on track, triggering its lane change termination trigger, at which point C_1 selects *end_lc* to conclude the manoeuvre, subsequently reaching a configuration satisfying Property 4. The algorithm therefore returns TRUE, along with the winning strategy tree shown as the double-bordered nodes in Fig. 5.

VI. COMPLEXITY AND CORRECTNESS PROOFS

The model checking algorithm has a worst-case time and space complexity exponential in the depth D of the time-abstract game tree, as derived below.

A. Termination

Termination can only be guaranteed via the time-limit or the maximum-steps termination conditions. The state-formula termination condition may never be satisfied and therefore does not guarantee termination. In many cases, the algorithm terminates early when a state satisfying the formula (in a reachability query) or a state violating the formula (in a safety query) is reached. In the following, we analyse termination guarantees for the remaining cases, in which no such state is encountered.

a) Maximum-steps M termination: Since the model checking algorithm traverses the game tree iteratively, termination under the maximum-steps (equivalently, maximum discrete-transitions) condition is straightforward: the game tree has a maximum depth of $D \in O(M)$, so the algorithm terminates after at most M recursive calls.

b) Time-limit T termination: In real-time systems, reaching the time limit in a finite number of steps must be guaranteed; the algorithm must therefore avoid *Zeno behaviour*, i.e., taking infinitely many transitions in a finite amount of time. HyTrig prevents this by ensuring that no agent $\alpha \in \text{Agt}$ takes the same action $a \in \text{Act}$ more than once within less than ϵ time units.

As a consequence, every sequence of at most $|\text{Agt}| \cdot |\text{Act}|$ consecutive transitions causes at least ϵ time units to elapse. Therefore, the number of transitions before the cumulative time exceeds T is bounded by

$$D \in O\left(\frac{T}{\epsilon} \cdot |\text{Agt}| \cdot |\text{Act}|\right),$$

which gives the order of the maximum depth of the game tree under time-limit termination.

B. Complexity

Since the algorithm performs a depth-first traversal of the time-abstract game tree, its worst-case complexity is proportional to the size of the tree. The branching factor of the tree is the maximum number of valid decisions available to the agents in any given state, equal to $|\text{Trig}| \cdot |\text{Act}|$, where $|\text{Trig}|$ is the total number of triggers (includes triggers of all agents) and $|\text{Act}|$ is the number of actions. The number of nodes in the tree is therefore

$$O((|\text{Trig}| \cdot |\text{Act}|)^D).$$

a) Space complexity: The algorithm stores an example or a counterexample strategy, which, in the worst case, spans the entire tree. In which case, the space complexity equals the tree size above.

b) Time complexity: Assuming the complexity of evaluating a state formula ψ is linear in the length of the formula $|\psi|$, the worst-case time complexity of the model checking algorithm is

$$O((|\text{Trig}| \cdot |\text{Act}|)^D \cdot |\psi|).$$

C. Correctness

In the two previous proofs, we established that the time-abstract game tree is finite. The model checking algorithm traverses through this game tree in order to find and construct the tree of a winning or a counter strategy for a given Hybrid ATL formula. The following theorem proves that the algorithm is sound and complete for reachability Hybrid ATL formulae. The proof for safety properties is symmetric. However, it is omitted due to space constraints.

Theorem VI.1. *Given an HGT $\mathcal{G}$*

$$\mathcal{G} := (X, \nu_0, \text{Agt}, \text{Act}, \text{Loc}, l_0, E, \text{trig}),$$

a reachability property $\phi \equiv \langle\langle A \rangle\rangle \mathbf{F} \psi$ for a set of agents $A \subseteq \text{Agt}$. Algorithm 1 produces a winning strategy for the formula ϕ iff a winning strategy, as defined in Def. V.2, exists.

Proof. Inductively, we prove each direction of the theorem separately.

a) *Soundness: the produced strategy is winning for ϕ :*

We show by induction on the depth d of the strategy tree that if Algorithm 1 returns TRUE for a decision node dn at depth d , then the strategy tree rooted at dn is a winning strategy tree for $\phi \equiv \langle\langle A \rangle\rangle \mathbf{F} \psi$.

Base case. ($d = 0$, i.e. $dn.config \models \psi$) (lines 2-8). The algorithm immediately attaches a property node as the sole child of dn and returns TRUE. The strategy tree contains a single leaf, the property node satisfying ψ . Therefore, every leaf satisfies ψ , and the strategy is winning by definition.

Inductive step. Assume the claim holds for all decision nodes at depth smaller than d . Suppose the algorithm returns TRUE for a decision node dn at depth d whose configuration does not satisfy ψ and is not terminal. Then there exists a time-child $tc \in children$ returned by $TimeChildren(dn, A)$ for which the algorithm returned TRUE. We consider two cases.

- *child is not a trigger node* (lines 12-16). The child is a property node satisfying ψ , and the algorithm returns TRUE directly. The leaf satisfies ψ , so the strategy is winning.
- *child is a trigger node controlled by a coalition agent* (lines 20-26) ($to-blame(child) \in A$). The algorithm found an action-child ac for which $EVALUATE(ac, \phi)$ returned TRUE. By the induction hypothesis, the strategy tree rooted at ac is winning. Since the coalition controls this trigger, it can choose the action leading to ac , guaranteeing ψ from ac onward. Hence, the strategy tree rooted at dn is winning.
- *child is a trigger node controlled by an adversarial agent* (lines 28-32) ($to-blame(child) \notin A$). The algorithm returned TRUE only after confirming that $EVALUATE(ac, \phi)$ returned TRUE for every action-child ac . By the induction hypothesis, every such subtree is winning. Since the adversary may choose any action, and every choice leads to a winning subtree, the strategy guarantees ψ regardless of the adversary's choice. Hence, the strategy tree rooted at dn is winning.

In all cases, every leaf of the resulting strategy tree is a property node satisfying ψ , so the produced strategy is winning for ϕ .

b) *Completeness: if a winning strategy exists, the algorithm finds one.* We show by induction on the depth d of a winning strategy tree $\mathcal{T}$ that if a winning strategy exists for ϕ from decision node dn , then Algorithm 1 returns TRUE for dn .

Base case ($d = 0$) (lines 2-8). If $dn.config \models \psi$, the algorithm returns TRUE immediately.

Inductive step. Assume the claim holds for all winning strategy trees of depth smaller than d . Let $\mathcal{T}$ be a winning strategy tree of depth d rooted at dn , whose configuration does not satisfy ψ and is not terminal. Since $\mathcal{T}$ is a strategy tree, its children are a subset of those returned by $TimeChildren(dn, A)$, so the children list is non-empty. Consider each child of dn that appears in $\mathcal{T}$ in order.

- *A property node child satisfying ψ* (lines 12-14). The algorithm encounters this child, returns TRUE, and terminates.
- *A trigger node controlled by a coalition agent* (lines 20-26). The winning strategy $\mathcal{T}$ designates at least one action-child ac whose subtree is winning and has depth smaller than d . By the induction hypothesis, $EVALUATE(ac, \phi)$ returns

TRUE. The algorithm iterates over action-children and will reach ac , returning TRUE.

- *A trigger node controlled by an adversarial agent* (lines 28-32). Since $\mathcal{T}$ is winning, every action-child of this trigger leads to a winning subtree of depth less than d . By the induction hypothesis, $EVALUATE$ returns TRUE for every action-child. The algorithm therefore does not return FALSE inside the adversarial loop and continues, eventually returning TRUE if this is the last child.

With these three cases, the induction is complete.

Combining items a) and b) gives both directions of the theorem's claim, i.e., the soundness and completeness of the model checking algorithm. $\square$

VII. EVALUATION

We evaluate HyTrig on three use cases: two scenarios illustrating model expressiveness and complex hybrid dynamics (a multi-lane highway traffic scenario and a robot volleyball game), and one benchmark from the literature [39], [40] (a cop-and-robber pursuit-evasion game).⁶ For each use case, we provide a game-generating script to allow users to easily experiment with different settings.

A. Case Study: Multi-Lane Highway

This case study builds on Example III-A, with one modification: we require that reachability strategies also satisfy the safety property. To achieve this, we place all cars in a single coalition for the reachability query,

$$\langle\langle C_1, \dots, C_N \rangle\rangle \mathbf{F} \bigwedge_{i=1}^N (x_i \geq x_i^0 + 10 \cdot v_{\max, i}), \quad (5)$$

and add the collision condition

$$\bigvee_{1 \leq i < j \leq N} (|x_i - x_j| < 5 \wedge |y_i - y_j| < 5)$$

as a termination condition, thereby ensuring that any generated reachability strategy remains safe. The main scaling factors are the number of cars and lanes; experimental configurations are outlined in Table I.

Results. The results shown in Table I are the average of multiple runs with randomly generated initial placements. The tool showed high performance up to 12 cars. In most cases, the initial placement of 10 cars on 4 lanes made the faster cars struggle to cover the required distance. Configurations beyond 12 cars did not terminate within 20 minutes, consistent with the exponential worst-case complexity established in Sect. VI-B; we manually terminated these runs rather than report partial results.

B. Case Study: Volleyball Game

This case study models a two-team robot volleyball game (cf. Fig. 6), showcasing adversarial coalitions and 3D projectile ball dynamics.

Model Configuration. The state space tracks continuous positions and velocities for players ($\{a, b, c, d\}$ of Fig. 6)

⁶All experiments are performed on a MacBook M1-Pro laptop.

N	L	$T(s)$	Strategy Tree Depth	Strategy Tree Size (nodes)	Result	Time (s)
6	3	60	20	20	True	1.994
8	4	60	20	20	True	2.8007
10	6	60	20	20	True	13.914
12	6	90	30	30	True	24.189
10	4	120	—	—	False	3.8775

TABLE I: Average HyTrig results for the multi-lane highway case study.

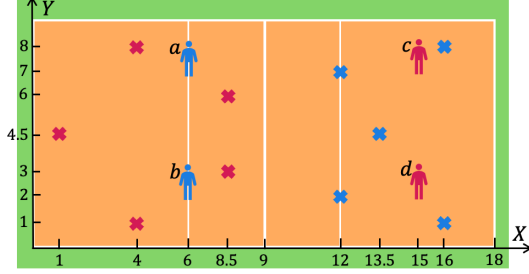


Fig. 6: Volleyball game with 2 players on each team and 5 target points for each court.

and the ball, alongside a discrete hitter variable htr . Agents utilise a number of target-directed hitting actions, along with *move* and *stop*. Movement triggers initiate positioning upon an opponent's hit, while hitting triggers enable the players to strike the ball at different proximity radii. The game terminates when the ball lands ($z_{ball} = 0$) or upon reaching a termination condition. The model is scalable on multiple dimensions: the number of players in each team, the number of hitting triggers, and the number of target points. Furthermore, the speed of each player significantly affects the outcome of the game and makes it last longer or shorter.

Properties. We analyse safety (not losing) and reachability (winning) properties. For team a, b , the safety property ensures the ball does not land on their court ($x < 9$):

$$\langle\langle a, b \rangle\rangle \mathbf{G} (x < 9 \rightarrow z > 0). \quad (6)$$

The reachability property requires forcing a ground hit on the opponent's side:

$$\langle\langle a, b \rangle\rangle \mathbf{F} (x > 9 \wedge z = 0). \quad (7)$$

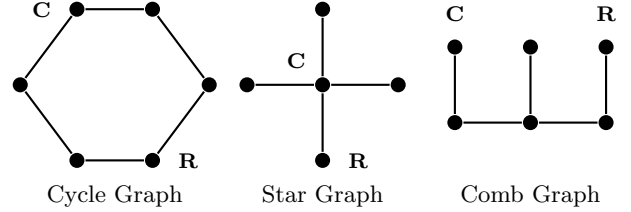
Symmetric properties hold for team c, d . Table II compares performance under multiple settings (number of players, hitting triggers, and target points).

Results. Table II confirms the exponential growth predicted in Sect. VI-B: the largest configuration reaches 572,375 strategy-tree nodes after only 10 discrete transitions. Among the tested factors, the number of agents has the strongest effect, growing the tree by up to 166 $\times$ when doubling players from 2 to 4 (block 1 vs. 4); triggers and target points have a comparable, smaller effect, each roughly growing the tree 4-6 $\times$ per increment (blocks 2-4).

The slowest runs in block 4 (538s and 554s, vs. 38s for the symmetric formulae) stem from Algorithm 1's asymmetric pruning: coalition-trigger nodes exit early on the first winning action, while adversarial-trigger nodes must exhaust all actions to confirm one, so queries that happen to require exhaustive

adversarial search cost far more than their structurally identical counterparts.

C. Benchmark Case Study: Cop-and-Robber

Fig. 7: Different graph structures for the cop-robber benchmark game with the initial location of the cop C and the robber R .

The cop-and-robber game [39], [40] is a pursuit game on a bidirectional graph in which the cop and the robber start at different nodes and move along the graph's edges. We adapt the metric variant [41], which assigns a length to each edge equal to the Euclidean distance between the corresponding points on a plane. The robber's speed is fixed at 1 distance unit per time unit, while the cop's speed is a parameter of the model. Fig. 7 shows common benchmark graph structures.

Model Configuration. The adaptation to an HGT is natural: the node coordinates serve as triggers for both the cop and the robber, and the actions allow each agent to choose which adjacent node to move to next. Flows for each agent are the normalised direction vectors of their selected edge. Invariants ensure all agents remain on the graph's edges.

Properties. The verification query checks whether the cop has a strategy to catch the robber, which may occur anywhere along an edge, not necessarily at a node:

$$\langle\langle \text{cop} \rangle\rangle \mathbf{F} (pos_{cop} = pos_{robber}). \quad (8)$$

Table III shows results for varying cop speed and graph structure and size. The time bound is fixed at $T = 90$ s.

Results. Even in a simple graph structure like a cycle, without heuristics, the algorithm exhaustively explores every path until either the time limit is reached or the cop's goal is achieved. This makes the example particularly challenging; nevertheless, the tool performs relatively well for up to 10 nodes. Note that tree Size reports only the retained winning strategy; runtime instead reflects the full search, including discarded branches, which is why size and time do not always correlate.

Across all use cases, the evaluation demonstrates two complementary strengths of HyTrig. The highway and volleyball

Formula	Tree Depth	Tree Size (nodes)	Result	Time (s)
2 players per team, 3 hitting triggers, 5 target points per court				
$\langle\langle a, b \rangle\rangle \mathbf{F}(x > 9 \wedge z = 0)$	—	—	FALSE	4.134
$\langle\langle c, d \rangle\rangle \mathbf{F}(x < 9 \wedge z = 0)$	—	—	FALSE	2.135
$\langle\langle a, b \rangle\rangle \mathbf{G}(x < 9 \rightarrow z > 0)$	22	3453	TRUE	2.186
$\langle\langle c, d \rangle\rangle \mathbf{G}(x > 9 \rightarrow z > 0)$	22	6755	TRUE	2.279
4 players per team, 1 hitting triggers, 3 target points per court				
$\langle\langle a, b \rangle\rangle \mathbf{F}(x > 9 \wedge z = 0)$	—	—	FALSE	3.539
$\langle\langle c, d \rangle\rangle \mathbf{F}(x < 9 \wedge z = 0)$	—	—	FALSE	2.570
$\langle\langle a, b \rangle\rangle \mathbf{G}(x < 9 \rightarrow z > 0)$	22	4485	TRUE	2.600
$\langle\langle c, d \rangle\rangle \mathbf{G}(x > 9 \rightarrow z > 0)$	22	6881	TRUE	1.781
4 players per team, 1 hitting triggers, 7 target points per court				
$\langle\langle a, b \rangle\rangle \mathbf{F}(x > 9 \wedge z = 0)$	—	—	FALSE	13.821
$\langle\langle c, d \rangle\rangle \mathbf{F}(x < 9 \wedge z = 0)$	—	—	FALSE	19.390
$\langle\langle a, b \rangle\rangle \mathbf{G}(x < 9 \rightarrow z > 0)$	22	24920	TRUE	19.436
$\langle\langle c, d \rangle\rangle \mathbf{G}(x > 9 \rightarrow z > 0)$	22	30666	TRUE	12.215
4 players per team, 2 hitting triggers, 5 target points per court				
$\langle\langle a, b \rangle\rangle \mathbf{F}(x > 9 \wedge z = 0)$	—	—	FALSE	38.707
$\langle\langle c, d \rangle\rangle \mathbf{F}(x < 9 \wedge z = 0)$	—	—	FALSE	538.382
$\langle\langle a, b \rangle\rangle \mathbf{G}(x < 9 \rightarrow z > 0)$	22	572375	TRUE	553.869
$\langle\langle c, d \rangle\rangle \mathbf{G}(x > 9 \rightarrow z > 0)$	22	97476	TRUE	37.773

TABLE II: HyTrig results for the volleyball case study, with the termination condition set to 10 discrete transitions.

Graph	Graph Nodes	Cop Speed	Tree Depth	Tree Size (nodes)	Result	Time(s)
Cycle	6	2	—	—	False	10.149
Cycle	6	2.5	30	407	True	7.840
Cycle	8	2.5	—	—	False	30.388
Star	7	2	26	1506	True	14.734
Star	9	2	26	1360	True	22.457
Comb	6	2	—	—	False	4.4
Comb	6	2.5	30	298	True	6.045
Comb	8	2.5	30	444	True	19.803
Comb	10	2.5	—	—	False	28.889

TABLE III: HyTrig results for the cop-and-robber benchmark verifying formula (8) at time-limit of 90 sec.

examples show that the trigger-based HGT framework expresses realistic, multi-agent hybrid scenarios with complex continuous and discrete dynamics, where the verification algorithm optimises search by terminating early upon discovering a winning strategy or decisive counterexample. The cop-and-robber benchmark example demonstrates that standard game-theoretic challenges transfer natively to the HGT formalism.

VIII. CONCLUSION

We present a bounded semantics for Hybrid Games with Triggers (HGT) that resolves semantic edge cases, such as deadlock conditions, left open by the original theoretical model [5], together with Hybrid ATL, an extension of alternating-time temporal logic with real-valued constraints for expressing winning conditions over bounded hybrid games. Building on these foundations, we develop a novel model-checking algorithm that verifies Hybrid ATL formulae by traversing the time-abstract game tree. By encoding agents' rationale into triggers, HGT effectively discretises infinite timing choices, rendering this verification decidable without restricting the system's continuous or discrete dynamics. We implement these contributions in HyTrig, a model checker that

leverages Julia's high-performance numerical solvers to handle complex bounded scenarios efficiently.

Future developments of HyTrig will focus on three key areas. First, we aim to optimise the model-checking algorithm by incorporating heuristics to guide the game-tree search and, where the underlying dynamics permit, leveraging reachability analysis techniques to prune or approximate parts of the state space. Second, we plan to broaden the tool's analytical reach by implementing additional verification algorithms, specifically for time-bounded monotonic hybrid systems [4] and ATL for discrete games with finite traces [42]. Third, we aim to enhance the user experience by expanding the GUI capabilities and supporting user-defined functions.

REFERENCES

- [1] T. A. Henzinger, B. Horowitz, and R. Majumdar, "Rectangular hybrid games," in *CONCUR '99: Concurrency Theory, 10th International Conference, Eindhoven, The Netherlands, August 24-27, 1999, Proceedings*, ser. Lecture Notes in Computer Science, J. C. M. Baeten and S. Mauw, Eds., vol. 1664. Springer, 1999, pp. 320–335. [Online]. Available: https://doi.org/10.1007/3-540-48320-9_23
- [2] T. A. Henzinger, P. W. Kopke, A. Puri, and P. Varaiya, "What's decidable about hybrid automata?" in *Proceedings of the Twenty-Seventh Annual ACM Symposium on Theory of Computing, 29 May-1 June 1995, Las Vegas, Nevada, USA*, F. T. Leighton and

- A. Borodin, Eds. ACM, 1995, pp. 373–382. [Online]. Available: <https://doi.org/10.1145/225058.225162>
- [3] T. Brihaye, L. Doyen, G. Geeraerts, J. Ouaknine, J.-F. Raskin, and J. Worrell, “On reachability for hybrid automata over bounded time,” in *Automata, Languages and Programming*, L. Aceto, M. Henzinger, and J. Sgall, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 416–427.
- [4] T. Brihaye, L. Doyen, G. Geeraerts, J. Ouaknine, J. Raskin, and J. Worrell, “Time-bounded reachability for monotonic hybrid automata: Complexity and fixed points,” in *Automated Technology for Verification and Analysis - 11th International Symposium, ATVA 2013, Hanoi, Vietnam, October 15-18, 2013. Proceedings*, ser. Lecture Notes in Computer Science, D. V. Hung and M. Ogawa, Eds., vol. 8172. Springer, 2013, pp. 55–70. [Online]. Available: https://doi.org/10.1007/978-3-319-02444-8_6
- [5] Q. Hamarneh and M. Schwammerger, “Trigger-based discretization of hybrid games for autonomous cyber-physical systems,” in *Proceedings of the 28th ACM International Conference on Hybrid Systems: Computation and Control, HSCC 2025, Irvine, CA, USA, May 6-9, 2025*. ACM, 2025, pp. 24:1–24:7. [Online]. Available: <https://doi.org/10.1145/3716863.3718032>
- [6] R. Alur, T. A. Henzinger, and O. Kupferman, “Alternating-time temporal logic,” *J. ACM*, vol. 49, no. 5, pp. 672–713, 2002. [Online]. Available: <https://doi.org/10.1145/585265.585270>
- [7] T. A. Henzinger, “The theory of hybrid automata,” in *Proceedings, 11th Annual IEEE Symposium on Logic in Computer Science, New Brunswick, New Jersey, USA, July 27-30, 1996*. IEEE Computer Society, 1996, pp. 278–292. [Online]. Available: <https://doi.org/10.1109/LICS.1996.561342>
- [8] R. Alur, T. A. Henzinger, F. Y. Mang, S. Qadeer, S. K. Rajamani, and S. Tasiran, “Mocha: Modularity in model checking,” in *International Conference on Computer Aided Verification*. Springer, 1998, pp. 521–525.
- [9] A. Lomuscio, H. Qu, and F. Raimondi, “MCMAS: an open-source model checker for the verification of multi-agent systems,” *Int. J. Softw. Tools Technol. Transf.*, vol. 19, no. 1, pp. 9–30, 2017. [Online]. Available: <https://doi.org/10.1007/s10009-015-0378-x>
- [10] T. Chen, V. Forejt, M. Kwiatkowska, D. Parker, and A. Simaitis, “Prism-games: A model checker for stochastic multi-player games,” in *International Conference on TOOLS and Algorithms for the Construction and Analysis of Systems*. Springer, 2013, pp. 185–191.
- [11] S. G. d. B. Garcia-Alcalde and F. Belardinelli, “Atl*: An automata-theoretic approach and tool for the verification of strategic abilities in multi-agent systems,” *arXiv preprint arXiv:2510.17306*, 2025.
- [12] R. Alur, “Timed Automata,” in *Computer Aided Verification*, N. Halbwachs and D. Peled, Eds. Berlin, Heidelberg: Springer, 1999, pp. 8–22.
- [13] G. Behrmann, A. Cougnard, A. David, E. Fleury, K. G. Larsen, and D. Lime, “Uppaal-tiga: Time for playing games! (tool paper),” in *International conference on computer aided verification*. Springer, 2007, pp. 121–125.
- [14] A. David, P. G. Jensen, K. G. Larsen, M. Mikučionis, and J. H. Taankvist, “Uppaal stratego,” in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2015, pp. 206–211.
- [15] P. W. Kopke, “The theory of rectangular hybrid automata,” Ph.D. dissertation, Cornell University, USA, 1996.
- [16] G. Lafferriere, G. J. Pappas, and S. Sastry, “O-minimal hybrid systems,” *Math. Control. Signals Syst.*, vol. 13, no. 1, pp. 1–21, 2000. [Online]. Available: <https://doi.org/10.1007/PL00009858>
- [17] P. Bouyer, T. Brihaye, and F. Chevalier, “O-minimal hybrid reachability games,” *Log. Methods Comput. Sci.*, vol. 6, no. 1, 2010. [Online]. Available: <http://arxiv.org/abs/0911.4833>
- [18] R. Alur, T. A. Henzinger, G. Lafferriere, and G. J. Pappas, “Discrete abstractions of hybrid systems,” *Proc. IEEE*, vol. 88, no. 7, pp. 971–984, 2000. [Online]. Available: <https://doi.org/10.1109/5.871304>
- [19] A. Platzer, “Differential dynamic logics,” *Künstliche Intell.*, vol. 24, no. 1, pp. 75–77, 2010. [Online]. Available: <https://doi.org/10.1007/s13218-010-0014-6>
- [20] —, “Differential game logic,” *CoRR*, vol. abs/1408.1980, 2014. [Online]. Available: <http://arxiv.org/abs/1408.1980>
- [21] A. Platzer and J. Quesel, “Keymaera: A hybrid theorem prover for hybrid systems (system description),” in *Automated Reasoning, 4th International Joint Conference, IJCAR 2008, Sydney, Australia, August 12-15, 2008. Proceedings*, ser. Lecture Notes in Computer Science, A. Armando, P. Baumgartner, and G. Dowek, Eds., vol. 5195. Springer, 2008, pp. 171–178. [Online]. Available: https://doi.org/10.1007/978-3-540-71070-7_15
- [22] N. Fulton, S. Mitsch, J. Quesel, M. Völz, and A. Platzer, “Keymaera X: an axiomatic tactical theorem prover for hybrid systems,” in *Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015. Proceedings*, ser. Lecture Notes in Computer Science, A. P. Felty and A. Middeldorp, Eds., vol. 9195. Springer, 2015, pp. 527–538. [Online]. Available: https://doi.org/10.1007/978-3-319-21401-6_36
- [23] T. A. Henzinger, P. Ho, and H. Wong-Toi, “HYTECH: A model checker for hybrid systems,” *Int. J. Softw. Tools Technol. Transf.*, vol. 1, no. 1-2, pp. 110–122, 1997. [Online]. Available: <https://doi.org/10.1007/s100090050008>
- [24] B. I. Silva and B. H. Krogh, “Formal verification of hybrid systems using checkmate: a case study,” in *American Control Conference, ACC 2000, Chicago, Illinois, USA, 28-30 June, 2000*. IEEE, 2000, pp. 1679–1683. [Online]. Available: <https://doi.org/10.1109/ACC.2000.879487>
- [25] G. Frehse, “Phaver: algorithmic verification of hybrid systems past hytech,” *Int. J. Softw. Tools Technol. Transf.*, vol. 10, no. 3, pp. 263–279, 2008. [Online]. Available: <https://doi.org/10.1007/s10009-007-0062-x>
- [26] G. Frehse, C. L. Guernic, A. Donzé, S. Cotton, R. Ray, O. Lebeltel, R. Ripado, A. Girard, T. Dang, and O. Maler, “Spacex: Scalable verification of hybrid systems,” in *Computer Aided Verification - 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14-20, 2011. Proceedings*, ser. Lecture Notes in Computer Science, G. Gopalakrishnan and S. Qadeer, Eds., vol. 6806. Springer, 2011, pp. 379–395. [Online]. Available: https://doi.org/10.1007/978-3-642-22110-1_30
- [27] P. S. Duggirala, M. Potok, S. Mitra, and M. Viswanathan, “C2E2: a tool for verifying annotated hybrid systems,” in *Proceedings of the 18th International Conference on Hybrid Systems: Computation and Control, HSCC’15, Seattle, WA, USA, April 14-16, 2015*, A. Girard and S. Sankaranarayanan, Eds. ACM, 2015, pp. 307–308. [Online]. Available: <https://doi.org/10.1145/2728606.2728646>
- [28] S. Kong, S. Gao, W. Chen, and E. M. Clarke, “dreach: δ -reachability analysis for hybrid systems,” in *Tools and Algorithms for the Construction and Analysis of Systems - 21st International Conference, TACAS 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings*, ser. Lecture Notes in Computer Science, C. Baier and C. Tinelli, Eds., vol. 9035. Springer, 2015, pp. 200–205. [Online]. Available: https://doi.org/10.1007/978-3-662-46681-0_15
- [29] X. Chen, E. Ábrahám, and S. Sankaranarayanan, “Taylor model flowpipe construction for non-linear hybrid systems,” in *Proceedings of the 33rd IEEE Real-Time Systems Symposium, RTSS 2012, San Juan, PR, USA, December 4-7, 2012*. IEEE Computer Society, 2012, pp. 183–192. [Online]. Available: <https://doi.org/10.1109/RTSS.2012.70>
- [30] S. Bogomolov, M. Forets, G. Frehse, K. Potomkin, and C. Schilling, “Juliereach: a toolbox for set-based reachability,” in *Proceedings of the 22nd ACM International Conference on Hybrid Systems: Computation and Control, HSCC 2019, Montreal, QC, Canada, April 16-18, 2019*, N. Ozay and P. Prabhakar, Eds. ACM, 2019, pp. 39–44. [Online]. Available: <https://doi.org/10.1145/3302504.3311804>
- [31] M. Kuderer, S. Gulati, and W. Burgard, “Learning driving styles for autonomous vehicles from demonstration,” in *IEEE International Conference on Robotics and Automation, ICRA 2015, Seattle, WA, USA, 26-30 May, 2015*. IEEE, 2015, pp. 2641–2646. [Online]. Available: <https://doi.org/10.1109/ICRA.2015.7139555>
- [32] K. G. Larsen, P. Pettersson, and W. Yi, “UPPAAL in a nutshell,” *Int. J. Softw. Tools Technol. Transf.*, vol. 1, no. 1-2, pp. 134–152, 1997. [Online]. Available: <https://doi.org/10.1007/s100090050010>
- [33] T. A. Henzinger and V. S. Prabhu, “Timed alternating-time temporal logic,” in *Formal Modeling and Analysis of Timed Systems, 4th International Conference, FORMATS 2006, Paris, France, September 25-27, 2006. Proceedings*, ser. Lecture Notes in Computer Science, E. Asarin and P. Bouyer, Eds., vol. 4202. Springer, 2006, pp. 1–17. [Online]. Available: https://doi.org/10.1007/11867340_1
- [34] T. Brihaye, F. Laroussinie, N. Markey, and G. Oreiby, “Timed concurrent game structures,” in *CONCUR 2007 - Concurrency Theory, 18th International Conference, CONCUR 2007, Lisbon, Portugal, September 3-8, 2007. Proceedings*, ser. Lecture Notes in Computer Science, L. Caires and V. T. Vasconcelos, Eds., vol. 4703. Springer, 2007, pp. 445–459. [Online]. Available: https://doi.org/10.1007/978-3-540-74407-8_30
- [35] J. Bezanson, S. Karpinski, V. B. Shah, and A. Edelman, “Julia: A fast dynamic language for technical computing,” *CoRR*, vol. abs/1209.5145, 2012. [Online]. Available: <http://arxiv.org/abs/1209.5145>
- [36] C. Rackauckas and Q. Nie, “DifferentialEquations.jl—a performant

- 1273 and feature-rich ecosystem for solving differential equations in julia,”
 1274 *Journal of open research software*, vol. 5, no. 1, pp. 15–15, 2017.
- 1275 [37] J. Zhang, K. H. Johansson, J. Lygeros, and S. Sastry, “Zeno hybrid
 1276 systems,” *International Journal of Robust and Nonlinear Control: IFAC-*
 1277 *Affiliated Journal*, vol. 11, no. 5, pp. 435–451, 2001.
- 1278 [38] C. Tsitouras, “Runge–kutta pairs of order 5 (4) satisfying only the
 1279 first column simplifying assumption,” *Computers & mathematics with*
 1280 *applications*, vol. 62, no. 2, pp. 770–775, 2011.
- 1281 [39] A. Quilliot, “Problemes de jeux, de point fixe, de connectivité et
 1282 de représentation sur des graphes, des ensembles ordonnés et des
 1283 hypergraphes,” *These d’Etat, Université de Paris VI*, pp. 131–145, 1983.
- 1284 [40] R. J. Nowakowski and P. Winkler, “Vertex-to-vertex pursuit in a
 1285 graph,” *Discret. Math.*, vol. 43, no. 2-3, pp. 235–239, 1983. [Online].
 1286 Available: [https://doi.org/10.1016/0012-365X\(83\)90160-7](https://doi.org/10.1016/0012-365X(83)90160-7)
- 1287 [41] D. Berend and M. D. Boshernitzan, “A cop-robber game on metric
 1288 graphs,” *arXiv preprint arXiv:2512.18468*, 2025.
- 1289 [42] F. Belardinelli, A. Lomuscio, A. Murano, and S. Rubin, “Alternating-
 1290 time temporal logic on finite traces,” in *Proceedings of the Twenty-*
 1291 *Seventh International Joint Conference on Artificial Intelligence, IJCAI*
 1292 *2018, July 13-19, 2018, Stockholm, Sweden*, J. Lang, Ed. ijcai.org, 2018,
 1293 pp. 77–83. [Online]. Available: <https://doi.org/10.24963/ijcai.2018/11>